UNIVERSIDAD CATÓLICA SANTO TORIBIO DE MOGROVEJO

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Sistema difuso de Mamdani

Docente:

Chavarry Chankay, Mariana

Integrantes:

Aguinaga Ramos, Estefany Pamela

Chacón Chavez Marco Antonio

Juárez Llatas, Nelson André

Pongo Calvay Victor Adolfo

2026 - I

ÍNDICE

1. Introducción y Justificación.....3
1.1. Definición del problema..... 3
1.2. Justificación..... 3
2. Diseño del Sistema Difuso..... 4
2.1. Variables Lingüísticas.....4
2.2. Conjuntos Difusos..... 5
2.3. Funciones de Membresía..... 7
3. Reglas Difusas y Motor de Inferencia..... 11
3.1. Base de Reglas..... 11
3.2. Motor de inferencia.....13
3.3. Método de defusificación..... 13
4. Implementación y Resultados.....13
4.1. Walkthrough de Código Limpio..... 13
4.2. Documentación de Pruebas.....14
4.3. Resultados Coherentes y Discusión.....15
5. Conclusiones.....16
6. Referencias.....16

1. Introducción y Justificación

1.1. Definición del problema

En los videojuegos de modo historia o peleas offline, los personajes no jugables (NPC) cumplen un rol fundamental en la interacción con el jugador y en la generación de desafíos. Sin embargo, muchos de estos NPC son controlados mediante técnicas tradicionales como árboles de decisión o reglas rígidas, lo que limita su capacidad de adaptación ante diferentes situaciones del entorno.

Esta rigidez provoca comportamientos poco realistas, predecibles y repetitivos afectando la experiencia del usuario. En particular, los NPC no logran reaccionar de manera gradual ante ante variables como la distancia al jugador o su propio nivel de salud, sino que responden de forma abrupta o binaria.

Por ello, surge la necesidad de implementar un sistema que permita modelar comportamientos más flexibles y naturales. En este contexto, el uso de la lógica difusa mediante el algoritmo de Mamdani se plantea como una solución adecuada para mejorar la toma de decisiones de los NPC, permitiendo respuestas más dinámicas y realistas.

1.2. Justificación

El desarrollo de los sistemas inteligentes en videojuegos es un área de creciente importancia, ya que influye directamente en la calidad de la experiencia del usuario. Los personajes no jugables (NPC) requieren comportamientos más realistas y adaptativos para responder de manera coherente a las acciones del jugador.

Las técnicas tradicionales, como los árboles de decisión, presentan limitaciones al manejar situaciones complejas o variables continuas, debido a su naturaleza rígida. En ese contexto, la lógica difusa se presenta como una alternativa eficaz, ya que permite trabajar con incertidumbre y representar el conocimiento mediante variables lingüísticas.

El uso del algoritmo Mamdani se justifica por su capacidad para modelar sistemas basados en reglas intuitivas del tipo “SI-ENTONCES”, facilitando la interpretación y el diseño del sistema. Además, permite generar respuestas

suaves y graduales, lo cual resulta ideal para simular comportamientos más naturales en los NPC.

2. Diseño del Sistema Difuso

2.1. Variables Lingüísticas

El sistema difuso implementado considera tres variables de entrada y una variables de salida, las cuales modelan el comportamiento del NPC dentro de un entorno de combate.

```
class VariableLinguistica:
  def __init__(self, nombre, universo, unidad=""):
      self.nombre    = nombre
      self.universo  = universo   # [min, max]
      self.unidad    = unidad
      self.conjuntos = {}        # nombre -> función de membresía

  def agregar_conjunto(self, nombre, fn):
      self.conjuntos[nombre] = fn
      return self

  def fusificar(self, x):
      """Devuelve {conjunto: grado} para el valor nítido x."""
      return {nombre: fn(x) for nombre, fn in self.conjuntos.items()}

  def __repr__(self):
      return f"VariableLinguistica({self.nombre}, conjuntos={list(self.conjuntos)})"
```

En la implementación del sistema difuso, se definió una clase denominada *LinguisticVar,* la cual permite representar las variables lingüísticas del modelo, incluyendo su universo de discurso y conjuntos difusos asociados.
Esta clase facilita la creación de variables como distancia, vida del NPC y vida del jugador, así como la asignación de sus respectivas funciones de membresía. Además, incorpora un método de fuzzificación que transforma valores numéricos en grados de pertenencia difusos

2.1.1. Variables de entrada

- Distancia de jugador (Distancia)
  - Representa proximidad entre el NPC y el jugador, en un rango de 0 a 100
- Nivel de vida del NPC (Vida_NPC)

- Indica el estado de salud del personaje controlado por el sistema, en un rango de 0 a 100

- Nivel de vida del jugador (Vida_jugador)
  - Representa la salud del oponente, también en un rango de 0 a 100

2.1.2. Variables de salida

- Acción del NPC (Acción)
  - Determina la decisión que tomará el NPC. Se representa en un rango del 1 al 100

2.2. Conjuntos Difusos

En el conjunto difuso propuesto, cada variable lingüística se descompone en un conjunto de etiquetas o estados cualitativos denominados conjuntos difusos, los cuales permiten interpretar valores numéricos de forma gradual. Estos conjuntos representan diferentes niveles o condiciones dentro del problema y están asociados a funciones de membresía que determinan el grado de pertenencia de un valor en específico.

2.2.1. Variable: Distancia al jugador

Universo de discurso: [0, 100]

- Baja
- Media
- Alta

Estos conjuntos permiten representar la proximidad entre el NPC y el jugador, lo cual influye directamente en la decisión del NPC.

2.2.1.1. Ejemplo de código

```
distancia = VariableLinguistica("Distancia", [0, 100], "u")

distancia.agregar_conjunto("Cerca", trapecio_izquierdo(20, 45))

distancia.agregar_conjunto("Media", triangulo(25, 50, 75))

distancia.agregar_conjunto("Lejos", trapecio_derecho(55, 80))
```

2.2.2. Variable: Vida del NPC

Universo de discurso: [0, 100]

- Baja
- Media
- Alta

Estos conjuntos representan el estado de salud del NPC, determinando su capacidad para tomar decisiones ofensivas o defensivas

2.2.2.1. Ejemplo en código

```
vida_npc = VariableLinguistica("Vida_NPC", [0, 100], "%")

vida_npc.agregar_conjunto("Baja",  trapecio_izquierdo(20, 40))

vida_npc.agregar_conjunto("Media", triangulo(25, 50, 75))

vida_npc.agregar_conjunto("Alta",  trapecio_derecho(60, 80))
```

2.2.3. Variable: Vida del Jugador

Universo de discurso: [0, 100]

- Baja
- Media
- Alta

Esta variable permite evaluar el estado del oponente, influyendo en la estrategia del NPC

2.2.3.1. Ejemplo en código

```
vida_jugador = VariableLinguistica("Vida_Jugador", [0, 100], "%")

vida_jugador.agregar_conjunto("Baja",  trapecio_izquierdo(20, 40))

vida_jugador.agregar_conjunto("Media", triangulo(25, 50, 75))

vida_jugador.agregar_conjunto("Alta",  trapecio_derecho(60, 80))
```

2.2.4. Variable de salida: Acción del NPC

Universo de discurso [0, 100]

- Alejarse
- Defender
- Acercarse
- Atacar

Estos conjuntos representan las posibles decisiones que puede tomar el NPC en función de la situación evaluada por el sistema difuso

2.2.4.1. Ejemplo en código

```
# ────────────────────────────────────────────────────────────
# VARIABLE LINGÜÍSTICA — SALIDA
# ────────────────────────────────────────────────────────────
accion = VariableLinguistica("Accion", [0, 100], "nivel")
accion.agregar_conjunto("Alejarse",  trapecio_izquierdo(15, 30))
accion.agregar_conjunto("Defender",  triangulo(20, 35, 50))
accion.agregar_conjunto("Acercarse", triangulo(40, 60, 80))
accion.agregar_conjunto("Atacar",    trapecio_derecho(70, 85))
```

2.3. Funciones de Membresía

Las funciones de membresía son funciones matemáticas que permiten determinar el grado de pertenencia de un valor numérico a un conjunto difuso, tomando valores en el intervalo [0,1].
En el presente sistema difuso, se emplearon funciones de tipo triangular y trapezoidal, debido a su simplicidad y eficiencia computacional, así como por su capacidad para modelar transiciones suaves entre los diferentes estados de las variables.
Estas funciones permiten que un mismo valor pertenezca parcialmente a varios conjuntos difusos, lo cual resulta fundamental para generar decisiones más naturales en el comportamiento del NPC

2.3.1. Tipos de funciones utilizadas

2.3.1.1. Función Triangular

- Se utiliza principalmente para representar estados intermedios (por ejemplo: “media”)

- Tiene la forma de un triángulo definido por tres puntos:
  - Inicio
  - Pico (máxima pertenencia = 1)
  - Fin
- Permite una transición gradual entre dos conjuntos extremos

2.3.1.2. Función Trapezoidal

- Se utiliza para representar estados extremos (por ejemplo: “baja”, “alta”, “cerca”, “lejos”)
- Está definida por cuatro puntos y presenta una zona donde la pertenencia es máxima (igual a 1), lo que permite modelar rangos más estables

2.3.1.3. Aplicaciones en las variables del sistema

1. Las funciones de membresía fueron definidas para cada variable de la siguiente manera
   - Distancia del jugador
     - Cerca => Función Trapezoidal (Izquierda)
     - Media => Función Triangular
     - Lejos => Función Trapezoidal (Derecha)

- Vida del NPC
  - Baja => Función Trapezoidal
  - Media => Función Triangular
  - Alta => Función trapezoidal
- Vida del jugador
  - Baja => Función Trapezoidal
  - Media => Función Triangular
  - Alta => Función trapezoidal
- Acción del NPC
  - Alejarse => Función Trapezoidal
  - Defender => Función Triangular
  - Acercarse => Función triangular
  - Atacar => Función trapezoidal

2.3.1.4 Función de membresía en código

- Trap_left(a,b) => Trapecio izquierdo

```
def trapecio_izquierdo(a, b):
  """Trapecio izquierdo: 1 si x <= a, decrece linealmente hasta 0 en b."""
  def f(x):
    if x <= a: return 1.0
    if x < b:  return (b - x) / (b - a)
    return 0.0
  return f
```

  - Comportamiento
    - Si x es pequeño => pertenece totalmente (1)
    - Luego baja gradualmente
    - Después ya no pertenece
- Trapezoid(a , b , c , d) => Trapecio Completo

```python
def trapecio(a, b, c, d):
    """Trapecio general: sube de a→b, plano b→c, baja c→d."""
    def f(x):
        if x <= a or x >= d: return 0.0
        if x < b:  return (x - a) / (b - a)
        if x <= c: return 1.0
        return (d - x) / (d - c)
    return f
```

- Triangle(a ,b, c) => Triangular

```python
def triangulo(a, b, c):
    """Triángulo: sube de a a b, baja de b a c."""
    def f(x):
        if x <= a or x >= c: return 0.0
        if x < b:  return (x - a) / (b - a)
        if x == b: return 1.0
        return (c - x) / (c - b)
    return f
```

- Trap_right(a, b) => Trapecio derecho

```python
def trapecio_derecho(a, b):
    """Trapecio derecho: crece desde a, 1 si x >= b."""
    def f(x):
        if x >= b: return 1.0
        if x > a:  return (x - a) / (b - a)
        return 0.0
    return f
```

3. Reglas Difusas y Motor de Inferencia

3.1. Base de Reglas

"El diseño de la base de reglas para el comportamiento del NPC se estructuró buscando emular el razonamiento táctico humano durante un combate. Como señala López Turiégano [1], el uso de la lógica difusa en videojuegos de lucha permite manejar la imprecisión implícita en las variables de entrada (tales como la distancia al oponente o los niveles de salud), favoreciendo además que el comportamiento general del controlador sea altamente comprensible a nivel humano. En nuestro sistema, la combinación de las variables *Distancia*, *Vida NPC* y *Vida Jugador* genera un conjunto de reglas (IF-THEN) que evitan la rigidez de los condicionales clásicos, permitiendo que el NPC evalúe el riesgo de forma orgánica antes de decidir su próxima acción."

La base de reglas constituye el núcleo del sistema difuso, ya que define el comportamiento del sistema mediante un conjunto de reglas lingüísticas del tipo SI-ENTONCES (IF-THEN).
Estas reglas permiten relacionar las variables de entrada (distancia, vida del NPC y vida del jugador) con la variable de salida (acción del NPC), simulando el razonamiento humano en la toma de decisiones.
En el sistema implementado, se definieron un total de 27 reglas difusas, resultado de la combinación de los diferentes niveles de cada variables de entrada:

- 3 niveles de distancia
- 3 niveles de vida del NPC
- 3 niveles de vida del jugador

Esto genera:

- 3 x 3 x 3 = 27 combinaciones posibles.

3.1.1 Ejemplos de reglas:

```
# ── Reglas: Distancia CERCA ──────────────────────────

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Alta"), (vida_jugador, "Alta")], {accion: "Atacar"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Alta"), (vida_jugador, "Media")], {accion: "Atacar"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Alta"), (vida_jugador, "Baja")], {accion: "Atacar"})
```

```
sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Media"), (vida_jugador, "Alta")], {accion: "Atacar"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Media"), (vida_jugador, "Media")], {accion:
"Atacar"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Media"), (vida_jugador, "Baja")], {accion: "Atacar"})



sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Baja"), (vida_jugador, "Alta")], {accion:
"Defender"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Baja"), (vida_jugador, "Media")], {accion: "Atacar"})

sistema.agregar_regla([(distancia, "Cerca"), (vida_npc, "Baja"), (vida_jugador, "Baja")], {accion: "Atacar"})
```

3.2. Motor de inferencia

Para el procesamiento de las reglas se implementó un motor de inferencia basado en el algoritmo de Mamdani. Este enfoque es altamente recomendado en la literatura de diseño de videojuegos para resolver dinámicas complejas con múltiples variables de entrada simultáneas. Tal y como demuestran Hanif et al. [2] en su implementación de lógica difusa para el ajuste dinámico de mecánicas de juego, el algoritmo de Mamdani resulta altamente efectivo para calcular variaciones fluidas basadas en el estado en tiempo real del juego (como el tiempo o elementos restantes). De manera equivalente, nuestro motor evalúa el grado de pertenencia de cada regla de combate (ej. distancia cerca y vida baja) para ponderar todas las fuerzas tácticas concurrentes en un mismo *frame* o turn.

- Fuzzificación
  - Los valores de entrada (distancia, vida del NPC y vida del jugador) son transformados en grados de pertenencia mediante las funciones de membresía

```
    def fusificar(self, x):
      """Devuelve {conjunto: grado} para el valor nítido x."""
      return {nombre: fn(x) for nombre, fn in self.conjuntos.items()}
```

- Evaluación de reglas
  - Cada regla es evaluada utilizando el operador lógico AND, implementando mediante la función mínimo (min)

```
def _evaluar_clausula(clausula, membresias):
  """Evalúa una cláusula y devuelve su grado de membresía."""
  var, conj = clausula[0], clausula[1]
```

```
    negado    = len(clausula) == 3 and clausula[2] == "NOT"
    mu = membresias[var][conj]
    return 1.0 - mu if negado else mu

def _activacion_regla(antecedentes, op, membresias):
    """
    antecedentes : lista de cláusulas  [ (var, conj) | (var, conj, "NOT") ]
    op           : "AND" → min  |  "OR" → max
    """
    valores = [_evaluar_clausula(c, membresias) for c in antecedentes]
    return min(valores) if op == "AND" else max(valores)
```

- Inferir reglas
  - Los resultados de todas las reglas se combinan utilizando el operador máximo (max), generando un único conjunto difuso de salida.
  - El resultado final es un conjunto difuso que representa las posibles acciones del NPC con distintos grados de pertenencia

```
# ── Razonamiento ──────────────────────────────
def _razonar_reglas(self, var_salida, membresias):
    """Agrega alfas (máximo) de todas las reglas relevantes a var_salida."""
    agregados = {nombre: 0.0 for nombre in var_salida.conjuntos}
    for regla in self.reglas:
        if var_salida not in regla["cons"]:
            continue
        alpha = _activacion_regla(regla["ant"], regla["op"], membresias)
        conj  = regla["cons"][var_salida]
        agregados[conj] = max(agregados[conj], alpha)
    return agregados
```

3.3. Método de defusificación

Para obtener un valor numérico a partir del conjunto difuso resultante, se emplea el método de defusificación por centroide.

Este método calcula el centro de gravedad del área bajo la curva del conjunto difuso, proporcionando un valor representativo de la salida.

Este enfoque es ampliamente utilizado debido a su precisión y estabilidad.

```
# ── Defusificación (centroide) ──────────────

def _defusificar(self, var_salida, agregados):
    inf, sup = var_salida.universo
    paso   = (sup - inf) / self.resolucion
    num = den = 0.0
    x = inf
```

```
    while x <= sup:
        mu = max(
            min(alpha, var_salida.conjuntos[nombre](x))
            for nombre, alpha in agregados.items()
        )
        num += x * mu
        den += mu
        x   += paso
    return num / den if den != 0 else (inf + sup) / 2
```

4. Implementación y Resultados

4.1. Walkthrough de Código Limpio

El sistema difuso fue implementado en Python siguiendo una estructura modular y clara, permitiendo la definición flexible de variables lingüísticas, funciones de membresía y reglas difusas.

Primero, se definió la clase LinguisticVar, la cual encapsula cada variable del sistema, incluyendo su universo de discurso y los conjuntos difusos asociados. Esta clase también implementa el proceso de fuzzificación, transformando valores numéricos en grados de pertenencia.

Posteriormente, se implementaron las funciones de membresía (triangulo, trapecio_izquierdo, trapecio_derecho y trapecio), que permiten modelar los distintos niveles de cada variable de entrada y salida.

Luego, se definieron las variables del sistema:

- Distancia
- Vida del NPC
- Vida del jugador
- Acción (salida)

A cada variable se le asignaron sus respectivos conjuntos difusos.

El siguiente paso fue la construcción del sistema de inferencia, donde se agregaron las 27 reglas difusas utilizando estructuras del tipo:

```
sistema.add_rule([...], {...})
```

Finalmente, el sistema realiza:

1. Fuzzificación de entradas
2. Evaluación de reglas con operador mínimo (AND)
3. Agregación con operador máximo (OR)
4. Defusificación mediante el método del centroide

Este flujo permite obtener una decisión final del NPC en función del contexto del combate.

4.2. Documentación de Pruebas

Se realizaron diversas pruebas variando los valores de entrada para evaluar el comportamiento del sistema difuso.

Caso 1:

- Distancia: baja (cerca)
- Vida NPC: alta
- Vida jugador: baja
  Resultado: Acción ≈ Atacar
  ✔ Comportamiento esperado: el NPC aprovecha su ventaja.

Caso 2:

- Distancia: media
- Vida NPC: media
- Vida jugador: alta
  Resultado: Acción ≈ Acercarse o Defender
  ✔ Comportamiento lógico: el NPC evalúa antes de atacar.

Caso 3:

- Distancia: cerca
- Vida NPC: baja
- Vida jugador: alta
  Resultado: Acción ≈ Alejarse
  ✔ Comportamiento coherente: prioriza supervivencia.

Caso 4:

- Distancia: lejos
- Vida NPC: alta
- Vida jugador: baja
  Resultado: Acción ≈ Acercarse
  ✔ Estrategia ofensiva adecuada.

4.3. Resultados Coherentes y Discusión

Los resultados obtenidos muestran que el sistema difuso logra modelar decisiones más naturales en comparación con enfoques tradicionales.

A diferencia de los sistemas rígidos, el NPC:

- No cambia bruscamente de comportamiento
- Evalúa múltiples factores simultáneamente
- Genera respuestas intermedias (no binarias)

Esto permite simular un comportamiento más humano y estratégico.

Además, el uso del método de defusificación por centroide proporciona resultados estables y representativos.

Como posible mejora, se podría:

- Ajustar funciones de membresía
- Añadir más variables (ej. energía, entorno)
- Optimizar reglas para comportamientos más complejos

5. Conclusiones

El Sistema Difuso Mamdani implementado cumplió satisfactoriamente con el objetivo de modelar el comportamiento estratégico de un Personaje No Jugable (NPC) en un entorno de combate, superando la rigidez de los enfoques de toma de decisiones tradicionales. El diseño se fundamentó en tres variables de entrada (Distancia, Vida NPC, Vida Jugador) con 27 reglas lógicas, permitiendo una representación lingüística e intuitiva del conocimiento táctico. La selección del método de inferencia de Mamdani, junto con la defusificación por centroide, resultó crucial para generar respuestas suaves y estables. Los resultados de las pruebas demostraron que el NPC puede evaluar múltiples factores simultáneamente y producir decisiones graduales y coherentes, simulando así un comportamiento más natural y humano. Finalmente, el sistema ofrece una base sólida, con potencial de mejora mediante la adición de más variables de contexto (como energía o entorno) para optimizar su complejidad y realismo.

6. Referencias

[1] C. J. López Turiégano, "Diseño de nuevas técnicas evolutivas para videojuegos de lucha," Tesis de Máster, Dept. Inteligencia Artificial, Universidad Politécnica de Madrid, Madrid, España, 2015.

[2] F. N. Hanif, P. D. Kusuma, y R. A. Nugrahaeni, "Fuzzy Mamdani for the primary balloon shooter game," FAST Journal of Computer Engineering: Progress, Application and Technology, vol. 2, no. 1, pp. 1-11, Mayo 2023.